



Numpy 1D en Python

Objectifs

Après avoir terminé ce lab, vous serez capable de :

- Importer et utiliser la bibliothèque `numpy`
- Effectuer des opérations avec `numpy`

Table des matières

- [Préparation](#)
- [C'est quoi Numpy ?](#)
- [Opérations de base](#)

Préparation

```
In [ ]: # Importer la bibliothèque numpy
import numpy as np
```

C'est quoi Numpy ?

Un tableau numpy (Numpy Array) est similaire à une liste. Il est généralement de taille fixe et chaque élément est du même type. Nous pouvons utiliser une liste pour créer un tableau numpy :

```
In [ ]: # Créer une liste python
a = ["0", 1, "deux", "3", 4]

# Créer un tableau numpy
a = np.array([0, 1, 2, 3, 4])
```

Comme pour une liste, nous pouvons accéder à chaque élément via un index entre crochets :

```
In [ ]: print("a[0]:", a[0])
print("a[1]:", a[1])
```

Type

Si nous vérifions le type du tableau, nous obtenons `numpy.ndarray` :

```
In [ ]: type(a)
```

L'attribut `dtype` nous indique le type de données contenues dans le tableau :

```
In [ ]: a.dtype
```

Attributs importants

1. **size** : Le nombre d'éléments dans le tableau.
2. **ndim** : Le nombre de dimensions (le rang du tableau).
3. **shape** : Un tuple indiquant la taille du tableau dans chaque dimension.

```
In [ ]: print(a.size)
print(a.ndim)
print(a.shape)
```

Opérations de base

Addition de vecteurs

L'addition de vecteurs en numpy est très efficace :

```
In [ ]: u = np.array([1, 0])
v = np.array([0, 1])
z = u + v
z
```

Multiplication de vecteurs par un scalaire

```
In [ ]: y = np.array([1, 2])
z = 2 * y
z
```

Produit de Hadamard (Produit élément par élément)

```
In [ ]: u = np.array([1, 2])
v = np.array([3, 2])
z = u * v
z
```

Produit scalaire (Dot Product)

```
In [ ]: resultat = np.dot(u, v)
resultat
```

Fonctions universelles (Universal Functions)

Numpy fournit de nombreuses fonctions mathématiques qui s'appliquent à l'ensemble du tableau :

```
In [ ]: # Calculer la moyenne
moyenne = a.mean()

# Trouver la valeur maximale
maximum = a.max()

# Calculer le sinus de chaque élément
x = np.array([0, np.pi/2, np.pi])
y = np.sin(x)
y
```

Slicing (Découpage)

Nous pouvons extraire une partie d'un tableau numpy comme ceci :

```
In [ ]: b = np.array([10, 20, 30, 40, 50])
d = b[1:4]
d
```

Félicitations !

Vous avez terminé ce lab sur les bases de Numpy 1D.
